

Report conversation

MainActivity:

```
goViewModel.getRequestAuth().observe(this, r
    @Override
    public void onChanged(Boolean re
        Util.log("requested Auth: "
        if (requestAuth) {
            goViewModel.setRequestAu
            // do something
        }
    }
});
```

in GoViewModel

```
private MutableLiveData<Boolean> requestAuth

    public MutableLiveData<Boolean> getReque
        return requestAuth;
    }

    public void setRequestAuth(Boolean requ
        this.requestAuth.postValue(requestAu
    }
```

this seems to be a hack to trigger a callback when authentication is requested. what is the correct way of achieving the same?

Right, this code uses a `MutLiveData<Boolean>` in `GoViewModel` to trigger a callback in `MainActivity` when authentication is needed. It's a bit hacky since it uses a boolean flag for triggering. A cleaner approach is to use a more event-driven pattern. Here's a way to do it:

1. Define an Event Wrapper:

Create a generic `Event` class to wrap around your data. This helps in handling one-time events like navigation or showing a Snackbar.

java

 Copy code

```
public class Event<T> {
    private T content;
    private boolean hasBeenHandled = false;

    public Event(T content) {
        this.content = content;
    }

    public T getContentIfNotHandled() {
        if (hasBeenHandled) {
            return null;
        } else {
            hasBeenHandled = true;
            return content;
        }
    }

    public T peekContent() {
        return content;
    }
}
```

2. Update ViewModel:

Change `requestAuth` to use Event wrapper.

java

 Copy code

```
private MutableLiveData<Event<Boolean>> requestAuth = new MutableLiveData<Event<Boolean>>() {

    public MutableLiveData<Event<Boolean>> getRequestAuth() {
        return requestAuth;
    }

    public void setRequestAuth(Boolean requestAuth) {
```

```

        this.requestAuth.postValue(new Event<>(requestAuth));
    }

```

3. Update Observer in Activity:

Modify the observer to handle the `Event` type.

java

 Copy code

```

goViewModel.getRequestAuth().observe(this, new Observer<Event<Boolean>>() {
    @Override
    public void onChanged(Event<Boolean> requestAuthEvent) {
        Boolean requestAuth = requestAuthEvent.getContentIfNotHandled();
        if (requestAuth != null && requestAuth) {
            Util.log("Requested Auth: " + requestAuth);
            // do something
        }
    }
});

```

This approach ensures that the `requestAuth` event is only handled once and avoids the potential issue of being triggered multiple times due to configuration changes (like device rotation). It's more in line with the event-driven nature of LiveData.

why would it be triggered multiple times? how does device rotation trigger a mutablelivedata callback?

When a device rotation occurs in an Android app, it triggers a configuration change. This results in the recreation of the activity (in your case, `MainActivity`). When the activity is recreated, it re-subscribes to the `viewModel`. If your `viewModel` has a `MutableLiveData` that already holds a value, it will immediately trigger the observer in the newly created activity with the existing value.

Here's the sequence of events:

- 1. Initial State:** Your `MainActivity` is observing `requestAuth` in `GoViewModel`.
- 2. Value Set:** At some point, `requestAuth` is set to `true`.

3. **Observer Triggered:** The observer in `MainActivity` is notified, and you handle the authentication request.
4. **Device Rotates:** The user rotates the device, causing `MainActivity` to be destroyed and recreated.
5. **Re-subscription:** The new instance of `MainActivity` starts observing `requestAuth`.
6. **Immediate Callback:** Since `requestAuth` already holds `true`, the observer in the new `MainActivity` instance is immediately triggered with this value.

This can lead to unintended behaviors, like handling the authentication request again after a configuration change, even if it was already handled.

Using an `Event` wrapper around your data:

- Ensures that the observer is triggered only if the event hasn't been handled.
- Once the event is handled (i.e., `getContentIfNotHandled()` is called), it won't trigger the observer again for the same event, preventing duplicate actions on configuration changes like device rotation.

It's a way to distinguish between a new event and an existing value that's just being replayed to a new observer.

but we still use `Boolean`, even though it is simply an auth invocation. can we do better?

Absolutely. Since you're using a `Boolean` just as a trigger for an authentication request, you can simplify this by using a more explicit, event-based approach without the need for a `Boolean` or any actual data. Here's a refined approach:

1. Event Class without Data:

Create a simple event class that doesn't wrap around any data. This class is just to signify an event has occurred.

java

 Copy code

```
public class AuthRequestEvent {  
    private boolean hasBeenHandled = false;
```

```

    public boolean isHandled() {
        return hasBeenHandled;
    }

    public void markAsHandled() {
        this.hasBeenHandled = true;
    }
}

```

2. Update ViewModel:

Use `AuthRequestEvent` instead of a `Boolean`.

java

 Copy code

```

private MutableLiveData<AuthRequestEvent> requestAuthEvent = new MutableLiveData<>();

public LiveData<AuthRequestEvent> getRequestAuthEvent() {
    return requestAuthEvent;
}

public void requestAuth() {
    requestAuthEvent.setValue(new AuthRequestEvent());
}

```

3. Update Observer in Activity:

Handle the `AuthRequestEvent`.

java

 Copy code

```

goViewModel.getRequestAuthEvent().observe(this, new Observer<AuthRequestEvent>() {
    @Override
    public void onChanged(AuthRequestEvent event) {
        if (event != null && !event.isHandled()) {
            Util.log("Authentication Requested");
            event.markAsHandled();
            // Handle auth request
        }
    }
}

```

```
}  
});
```

In this revised approach:

- `AuthRequestEvent` serves as a clear signal for authentication requests.
- You avoid using a `Boolean`, which can be ambiguous and unnecessary.
- This method clearly communicates the purpose and ensures that each event is processed only once.

By doing this, your code becomes more readable and the intent of triggering an

```
... ..
```